关于 TSP 的 sealine 算法与模拟退火（2026-04-30）

这几天，发现算法代码有 BUG，有时有的点在分层时陷入无限循环中。于是，改 BUG 思考，疲劳，这种工作以后尽量少理，AI 的工作不是人做的乎。

sealine 算法的同心圆分层，也是有多种算法的，分层后，如何连成一条线，也是有多种算法的，sealine 的同心圆分层次后，就是普通组合算法了乎，以 sealine 的分层同心圆为外层最大架构，内部再应用诸多组合算法乎。

思考太疲劳，人不是 AI，人要无穷多事的，V 速退。

==========

关于 TSP 的 sealine 算法与模拟退火（2026-04-27）

今晚，我忽然想到，我设想的 sealine 算法，绝不相交线，大道理比模拟退火应该优胜，为何会在绝对规则矩形中比不上呢?

1）于是，修改下代码，8641 行将原先的 if curminH < minH then　改成 if curminH <= minH then　，加多一个等号，测试，三种坐标下，全比模拟退火优胜了。在坐标点是十分规则的矩形分布坐标轴一样整点时，一样 sealine 算法优胜了。

是不是我的模拟退火算法，参数设不对？这个不想理了。我认为，传统的遗传算法，蚁群算法，粒子群算法，模拟退火算法，以模拟退火为最优胜的。

2）至于 LKH，concorde，这两个即使开源代码也看不明，也不易对比，暂不理了。

3）随后更多的测试表明，有时仍然是模拟退火算法优胜，有时 sealine 算法优胜，相差好似不太大，但作为一种创新算法 sealine 算法，应该成立乎？！

刚才测试发现，有时，sealine(SL)算法居然出现一条相交线，这个不知代码哪里出了 BUG 了，这个查起来很烦人很疲劳，还是暂不理，V 速退了。

测试表明，SL 与 SA 确似各有优劣的。

刚才再测试发现，有一次随机数 300 点时，sealine 算法象死循环一样没反应没下文，估计可能有未知的代码 BUG，暂不理，烦，疲劳。。。

测试，三种坐标，

1）当坐标点是十分规则的矩形分布坐标轴一样整点时，将原先的 if curminH < minH t　改成 if curminH <= minH sealine 算法目前似乎平稳地优胜过模拟退火，就算点数从 100 到 500 都是一样

```
SQLQuery1.FieldByName('px').AsInteger := 100 + (100 * (i - 1) mod 1000);
SQLQuery1.FieldByName('py').AsInteger := 100 + 100 * (100 * (i - 1) div 1000);
```

2）当坐标点是随机分布，sealine 算法目前自认为确实在 100 点，到 500 点，比模拟退火效果各有优劣，高于 500 点时，个人 PC 电脑硬件问题，不理了。

```
SQLQuery1.FieldByName('px').AsInteger := Random(1000);//100 + I * 10;
SQLQuery1.FieldByName('py').AsInteger := Random(1000);//100 + I * 10;
```

3）当坐标点是大体规则矩形加上少量随机分布，sealine 算法目前仍然自认为确实在 100 点，到 500 点，比模拟退火效果要好不少，高于 500 点时，个人 PC 电脑硬件问题，不理了。

```
SQLQuery1.FieldByName('px').AsInteger :=100 + (100 * (i - 1) mod 1000) + Random(20);
SQLQuery1.FieldByName('py').AsInteger :=100 + 100 * (100 * (i - 1) div 1000) + Random(20);
```

设想点数越多越趋于最优解，这个估计可以这样认为，当点数越多时，点的分布就会象晶体一样有序，一个紧挨一个，象标准矩形，坐标轴坐标点一样，所以，当点数越来越多时，sealine 算法可能理论上是趋于最优解乎？？？

＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝

关于 TSP 的 sealine 算法与角谷猜想（2026-04-25）

1）TSP 的 sealine 算法已基本完成代码，测试表明，可能点数越多越趋于精确解，原本来于想法：真空包装，真空包装塑料薄膜，中间抽空气，内陷，被内部质点顶点顶住，这是立体三维，再降维成二维平面时，包装塑料薄膜变换成橡皮筋。实际上，写代码已经最简单实现功能，但达不到模拟模型的理想状态，这是一个可以发展的方向乎，暂称为，量子潮汐涨落模拟－真空包装－塑料薄膜－橡皮筋－海岸线模型猜想，简称 sealine 算法/猜想，只是实现最简单代码乎。。。

2) 角谷猜想熵增模拟相似，先连续 1 称为 I，连续 0 称为 O，字母 IO 代替数字 10，然后边界处开始模糊光栅衍射，连续 1 连续 0 递减，最后 10 相邻，即由 IO 相邻变成 10 相邻。

2026-04-25 今日很不容易，忍住疲劳，终于，修正了原有的 sealine 算法的 bug，测试下，比较满意了。

测试表明，肯定不是最优解，最优解目前只有动态规划或穷举算法，暂无解。
传统近似算法，个人目前认为，可能模拟退火算法是最好的，于是对比。
原本认为，sealine 算法，点数少时不及传统近似算法，设想点数越多越趋于最优解，这个估计可以这样认为，当点数越多时，点的分布就会象晶体一样有序，一个紧挨一个，象标准矩形，坐标轴坐标点一样，于是测试。
发现，
1）当坐标点是十分规则的矩形分布坐标轴一样整点时，sealine 算法目前不及模拟退火，就算点数从 100 到 500 都是一样

```
SQLQuery1.FieldByName('px').AsInteger := 100 + (100 * (i - 1) mod 1000);
SQLQuery1.FieldByName('py').AsInteger := 100 + 100 * (100 * (i - 1) div 1000);
```

2）当坐标点是随机分布，sealine 算法目前自认为确实在 100 点，到 500 点，比模拟退火效果要好不少，高于 500 点时，个人 PC 电脑硬件问题，不理了。

```
SQLQuery1.FieldByName('px').AsInteger := Random(1000);//100 + I * 10;
SQLQuery1.FieldByName('py').AsInteger := Random(1000);//100 + I * 10;
```

3）当坐标点是大体规则矩形加上少量随机分布，sealine 算法目前仍然自认为确实在 100 点，到 500 点，比模拟退火效果要好不少，高于 500 点时，个人 PC 电脑硬件问题，不理了。

```
SQLQuery1.FieldByName('px').AsInteger :=100 + (100 * (i - 1) mod 1000) + Random(20);
SQLQuery1.FieldByName('py').AsInteger :=100 + 100 * (100 * (i - 1) div 1000) + Random(20);
```

sealine 算法，目前不想理了，暂如是，停鞍稍驻初程，写代码象古代骑马驿站赶路，太急不好，疲劳不好。。。。。。。

TSP_ubuntu_Lazarus 源代码=033=sealine=粗略已处理四正四隅垂直和水平.zip
(1) https://github.com/aMeTooFor/TSP/
(2) https://blog.csdn.net/e271828/article/details/160509634?spm=1011.2124.3001.6209
＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝
SeaLine 算法的思考进展
2026-03-31
1）昨天我测试了一下，当 sealine 算法逐层法只有一层时，测试表明，可能正是最优解，但未获严格证明
2）昨天我又测试一下，将 sealine 算法选逐层法再逐点法相结合时，如果第二层只有一个点，共只有两层，测试表明，可能也是最优解，但未证明
3）昨天我又设想，将逐点法，由原来的最小距离/最大夹角/最小面积，改成了，最小边长，min(ac+bc-ab),测试表明，结果与原有的模拟退火相近似，也是近似解，但比原来的改进了许多。
4）我又测试下，逐层法后再逐点法，未写代码，三重循环还是二重循环，疲劳分不清，暂不写代码，等不疲劳分清楚再说
5）不写新代码，原有的测试，逐层法后再逐点法，以二层测试，有时相差不远，有时相差远，可能与第二层点的分布有关。可能是一种趋近近似算法。也可能要以改良成精确最优解，只是目前做不到。
6）TSP 的穷举法算法复杂度或为 O(n!)？今如果转化为 sealine 逐层法 n=L=L0+L1+L2+...+Li+...+Ln，是不是每层都要全排序呢？O(L1!+L2!+...+Li!+L...+Ln!)如果这样，也没甚用了，因为，动态规划法也是仅有 O(n*2^n)和 O(n^2*2n)。转化为，每层逐点时，要按原来的同心圆的顺序，不可乱来，分顺时针和逆时针，所以，可能 sealine 逐层且逐点法的复杂度为线性的 O(3n+1)，是有可能的。且不断逼近最优解，如果想下就可以了。。。。
＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝＝
TSP 海岸线 SeaLine 算法的逐层法/逐点法（续）
三界火宅人　　2026-03-26
假设 TSP 的海岸线 sealine 算法逐层产生同心圆的线圈所在点集为 L0,L1,L2...Li,...Ln,如果用递增方式，由集合前段 L0-->L0+L1-->L0+L1+L2-->L0+L1+...Li，这样来求 TSP 的精确解，会不会有简易算法，比直接全集 L 的 TSP 组合算法要简单?
1）若仅一层 L=L0，sealine 算法是不是最优解呢?
2）若仅两层 L=L0+L1，是否存在简易算法，使得由 L0-->L0＋L1 计算 TSP 最优解比直接 L 要简易得多呢?
3）如果上面 1）和 2）成立，根据数学归纳法，P=NP 仍然有可能成立的乎。一生二，二生三，三生万物者，数学归纳法乎。之乎者也，知苦这也。
4）由第一层向第二层推进，再分解分析降解为第一层内部加多一个点乎，这更加简化，算法如何？如果新点近边靠边圈，可能真的局部调下即成乎，如果新点近中心，可能大变， 不是这么简化的？P，线性乎，NP，级数指数乎，仍是未知的。

5）角谷猜想的熵增定律的映像，仍然可能成立乎
6）代数数理（辟支？知乎）与几何直观（声闻？福乎）不可能相差太远乎，自然数平方的倒数和，欧拉用级数解，现代化为几何直观圆，总有几何对应代数乎，素数对应几何上啥呢?
==========

关于 TSP 的海岸线猜想：SeaLine 算法的逐层法（不同于逐点法）

三界火宅人　　2026-03-25

经过两天的不充分休息，疲劳暂解，又开始强迫症式地新思想了，忽想到，如果海浪线不是每次一点点地强逼进，不理什么最近距离或最大夹角或最小面积，改用同心圆的方式，一线线地强逼进，最后再连通各个同心圆，这算法可行否?
1）为了一个想法，又开始焦虑不安了，必要调试出个结果才安心也，心无奎碍，无挂疑故，才可以安心休息也。
于是，又是上机调试，又写自定义函数，一个数组下标出错，也找很久，当前下标是 curA 还是 i，自已易搞混，原先正常的函数 isCross 或 ifCross，为啥又出错了，深入内部代码调试一番，这又累呀。一大早起床，啥也不干，就开机，没有报酬的，白干的，就为了一个想法，一个知字乎，为了讨个说法乎?
2）首先得到最外层的围城的围线算法，以前是一个点一个点地推进，现在改了，把围城推倒，相当于重新造一个围墙，把原先围城的点暂时不要。
3）就这递归，叠代，一线一线地推进，产生同心圆，最后连通这些同心圆，可行不?
结果，可行，但是，仍然不是最优解，不过没有细长深长的线了，比较钝角了。
没想到，不许相交，从这点出发的 SeaLine 算法，的确可行实现了，但离最优解差得远，不及有些相交线的组合算法，就这样，暂时算（蒜子）了，V 速退
==========

```
function TMain.sealine000(minx, miny, maxx, maxy: integer): integer;
var
  i, ijk: integer;      //memo_seaLine
  curp, curp0: integer;//TPoint;
  pline: TlinePoint;
  pcode1, pcode2: string;
  pcode1Shape, pcode2Shape: tshape;
  pcode1Pointer, pcode2Pointer: tpoint;
  path, ss: string;
  minpx, minpy, maxpx, maxpy: integer;
  sortlist, floorLine: TStringList;
  k1, k2, a, b, c, d: double;
begin
  sortlist := TStringList.Create;
  floorLine := TStringList.Create;
  sortlist.Clear;
  floorLine.Clear;
  ////////////////////////////////////四正　子午卯酉
  //minpx,minpy,maxpx,maxpy:integer;
  minpx := TPointLine(PointList[minx]).px;
  minpy := TPointLine(PointList[miny]).py;
  maxpx := TPointLine(PointList[maxx]).px;
  maxpy := TPointLine(PointList[maxy]).py;
  ///#####################################
  sortlist.Clear;
  for i := 0 to PointList.Count - 1 do
    //for i := PointList.Count - 1 downto 0 do
  begin
    //if    TPointLine(PointList[i]).floor<>-123 then
    if (TPointLine(PointList[i]).px = TPointLine(PointList[minx]).px) then
      // if  (TPointLine(PointList[i]).py <TPointLine(PointList[minx]).py) then
    begin
```

```
    //  minx:=i;
    // TPointLine(PointList[i]).floor:=-123;
    //sortlist.add(inttostr(TPointLine(PointList[i]).py)+'              '+TPointLine(PointList[i]).pcode);
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py), 10) +
      '              ' + IntToStr(i));
    application.ProcessMessages;
   end;
 end;
//ss:=sortlist.text;
sortlist.sort;
//ss:=sortlist.text;
//for i := 0 to sortlist.Count - 1 do
for i := sortlist.Count - 1 downto 0 do
  floorLine.add(trim(rightstr(sortlist[i], 10)));
//四正：酉正，从酉初到酉末
ss := floorLine.Text;
if sortlist.Count > 0 then
  minx := StrToInt(trim(rightstr(sortlist[0], 10)));     //酉时末，顶上
///#####################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).py = TPointLine(PointList[miny]).py) then
  begin
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px), 10) +
      '              ' + IntToStr(i));
    application.ProcessMessages;
  end;
end;
sortlist.sort;
if sortlist.Count > 0 then
  miny := StrToInt(trim(rightstr(sortlist[0], 10)));//子时初
application.ProcessMessages;
curp := minx;
curp0 := -1;
path := '';
while curp0 <> curp do//八极之四隅之亥，从酉末到子初也
begin
  //if curp<>-1 then
  application.ProcessMessages;
  curp0 := curp;
  for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
      if (TPointLine(PointList[i]).px < TPointLine(PointList[curp0]).px) then
        continue;
      if (TPointLine(PointList[i]).py > TPointLine(PointList[curp0]).py) then
        continue;
      if (TPointLine(PointList[i]).py > TPointLine(PointList[minx]).py) then
        continue;
      if (TPointLine(PointList[i]).px <= TPointLine(PointList[minx]).px) then
```

```
  continue;
if (TPointLine(PointList[i]).py <= TPointLine(PointList[miny]).py) then
  continue;
if (TPointLine(PointList[i]).px > TPointLine(PointList[miny]).px) then
  continue;
//if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
// if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
if (i = curp0) then
  continue;
if (i = miny) then
  continue;
if (i = minx) then
  continue;
if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
begin        //k1,k2,a,b,c,d:double;
  a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
  b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
  c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
  d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
  //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
  //if ((TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py) /
  //  (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px)) <
  //  ((TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py) /
  //  (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px)) then
  if ((b = 0) and (d <> 0)) then
  begin
    continue;
  end;
  if ((b <> 0) and (d = 0)) then
  begin
    curp := i;
  end;
  if ((b = 0) and (d = 0)) then
  begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
    if TPointLine(PointList[i]).py > TPointLine(PointList[curp]).py then
      //这里本可以加等号，也可不要等号
      curp := i;
  end;
  if ((b <> 0) and (d <> 0)) then
    if ((a * d) = (b * c)) then
      //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
    begin
      //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
      if TPointLine(PointList[i]).py > TPointLine(PointList[curp]).py then
        //这里本可以加等号，也可不要等号
        curp := i;
    end
    else if ((a * d) < (b * c)) then
      //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
    begin
      curp := i;
```

```
        end;
      end
      else     //第一次
       curp := i;
    end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := 0;
    TPointLine(PointList[curp]).floor := 0;
  end;
end;
if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  floorLine.add(IntToStr(curp));
// memo_seaLine.Lines.Add('1:' + path);
application.ProcessMessages;
////////////////////////////////////////////////////////////////
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).py = TPointLine(PointList[miny]).py) then
  begin
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px), 10) +
      '              ' + IntToStr(i));
    application.ProcessMessages;
  end;
end;
ss := sortlist.Text;
sortlist.sort;
for i := 0 to sortlist.Count - 1 do
  // for i := sortlist.Count - 1 downto 0 do
  floorLine.add(trim(rightstr(sortlist[i], 10)));//四正：子正，从子初到子末
// miny := StrToInt(sortlist[0]);
// if floorLine.IndexOf(IntToStr(miny)) <= -1 then
//   floorLine.add(IntToStr(miny));
if sortlist.Count > 0 then
  miny := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));
//子时末，右侧
///#####################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).px = TPointLine(PointList[maxx]).px) then
  begin
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py), 10) +
      '              ' + IntToStr(i));
    application.ProcessMessages;
  end;
```

```
end;
ss := sortlist.Text;
sortlist.sort;
if sortlist.Count > 0 then
  //maxx := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));//卯时初
  maxx := StrToInt(trim(rightstr(sortlist[0], 10)));//卯时初
////////////////////////////////////
application.ProcessMessages;
curp := miny;
curp0 := -1;
path := '';
while curp0 <> curp do //八极之四隅之寅，从子末到卯初
begin
  application.ProcessMessages;
  curp0 := curp;
  for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
      if (TPointLine(PointList[i]).px < TPointLine(PointList[curp0]).px) then
        continue;
      if (TPointLine(PointList[i]).py < TPointLine(PointList[curp0]).py) then
        continue;
      if (TPointLine(PointList[i]).py > TPointLine(PointList[maxx]).py) then
        continue;
      if (TPointLine(PointList[i]).px >= TPointLine(PointList[maxx]).px) then
        continue;
      if (TPointLine(PointList[i]).py <= TPointLine(PointList[miny]).py) then
        continue;
      if (TPointLine(PointList[i]).px < TPointLine(PointList[miny]).px) then
        continue;
      //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
      // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
      if (i = curp0) then
        continue;
      if (i = miny) then
        continue;
      if (i = maxx) then
        continue;
      if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
      begin       //k1,k2,a,b,c,d:double;
        a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
        b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
        c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
        d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
        //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        //if ((TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py) /
        //  (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px)) <
        //  ((TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py) /
        //  (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px)) then
        if ((b = 0) and (d <> 0)) then
        begin
```

```
        continue;
      end;
      if ((b <> 0) and (d = 0)) then
      begin
        curp := i;
      end;
      if ((b = 0) and (d = 0)) then
      begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
        if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
          //这里本可以加等号，也可不要等号
          curp := i;
      end;
      if ((b <> 0) and (d <> 0)) then
        if ((a * d) = (b * c)) then
          //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        begin
          //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
          if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
            //这里本可以加等号，也可不要等号
            curp := i;
        end
        else if ((a * d) < (b * c)) then
          //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        begin
          curp := i;
        end;
      end
      else     //第一次
        curp := i;
    end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := 0;
    TPointLine(PointList[curp]).floor := 0;
  end;
end;
if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  floorLine.add(IntToStr(curp));
//  memo_seaLine.Lines.Add('2:' + path);
application.ProcessMessages;
////////////////////////////////////////////////////////////////
///#######################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).px = TPointLine(PointList[maxx]).px) then
  begin
```

```
   sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py), 10) +
     '              ' + IntToStr(i));
   application.ProcessMessages;
  end;
end;
ss := sortlist.Text;
sortlist.sort;
for i := 0 to sortlist.Count - 1 do
  // for i := sortlist.Count - 1 downto 0 do
  floorLine.add(trim(rightstr(sortlist[i], 10)));//四正：卯正，从卯初到卯末
// maxx := StrToInt(sortlist[0]);
// if floorLine.IndexOf(IntToStr(maxx)) <= -1 then
//   floorLine.add(IntToStr(maxx));
if sortlist.Count > 0 then
  maxx := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));
//卯时末，右底

sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).py = TPointLine(PointList[maxy]).py) then
  begin
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px), 10) +
      '              ' + IntToStr(i));
    application.ProcessMessages;
  end;
end;
sortlist.sort;
ss := sortlist.Text;
if sortlist.Count > 0 then
  maxy := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));//午时初
///#######################################
////////////////////////////////////
ss := floorLine.Text;
application.ProcessMessages;
curp := maxx;
curp0 := -1;
path := '';
while curp0 <> curp do//八极之四隅之巳，从卯末到午初
begin
  application.ProcessMessages;
  curp0 := curp;
  //for i := 0 to PointList.Count - 1 do
  for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
      if (TPointLine(PointList[i]).px > TPointLine(PointList[curp0]).px) then
        continue;
      if (TPointLine(PointList[i]).py < TPointLine(PointList[curp0]).py) then
        continue;
      if (TPointLine(PointList[i]).py < TPointLine(PointList[maxx]).py) then
```

```
  continue;
 if (TPointLine(PointList[i]).px >= TPointLine(PointList[maxx]).px) then
  continue;
 if (TPointLine(PointList[i]).py >= TPointLine(PointList[maxy]).py) then
  continue;
 if (TPointLine(PointList[i]).px < TPointLine(PointList[maxy]).px) then
  continue;
 //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
 // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
 if (i = curp0) then
  continue;
 if (i = maxy) then
  continue;
 if (i = maxx) then
  continue;
 if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
 begin       //k1,k2,a,b,c,d:double;
  a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
  b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
  c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
  d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
  //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
  //if ((TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py) /
  //  (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px)) <
  //  ((TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py) /
  //  (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px)) then
  if ((b = 0) and (d <> 0)) then
  begin
   continue;
  end;
  if ((b <> 0) and (d = 0)) then
  begin
   curp := i;
  end;
  if ((b = 0) and (d = 0)) then
  begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
   if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
    //这里本可以加等号，也可不要等号
    curp := i;
  end;
  if ((b <> 0) and (d <> 0)) then
   if ((a * d) = (b * c)) then
    //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
   begin
    //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
    if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
     //这里本可以加等号，也可不要等号
     curp := i;
   end
   else if ((a * d) < (b * c)) then
    //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
```

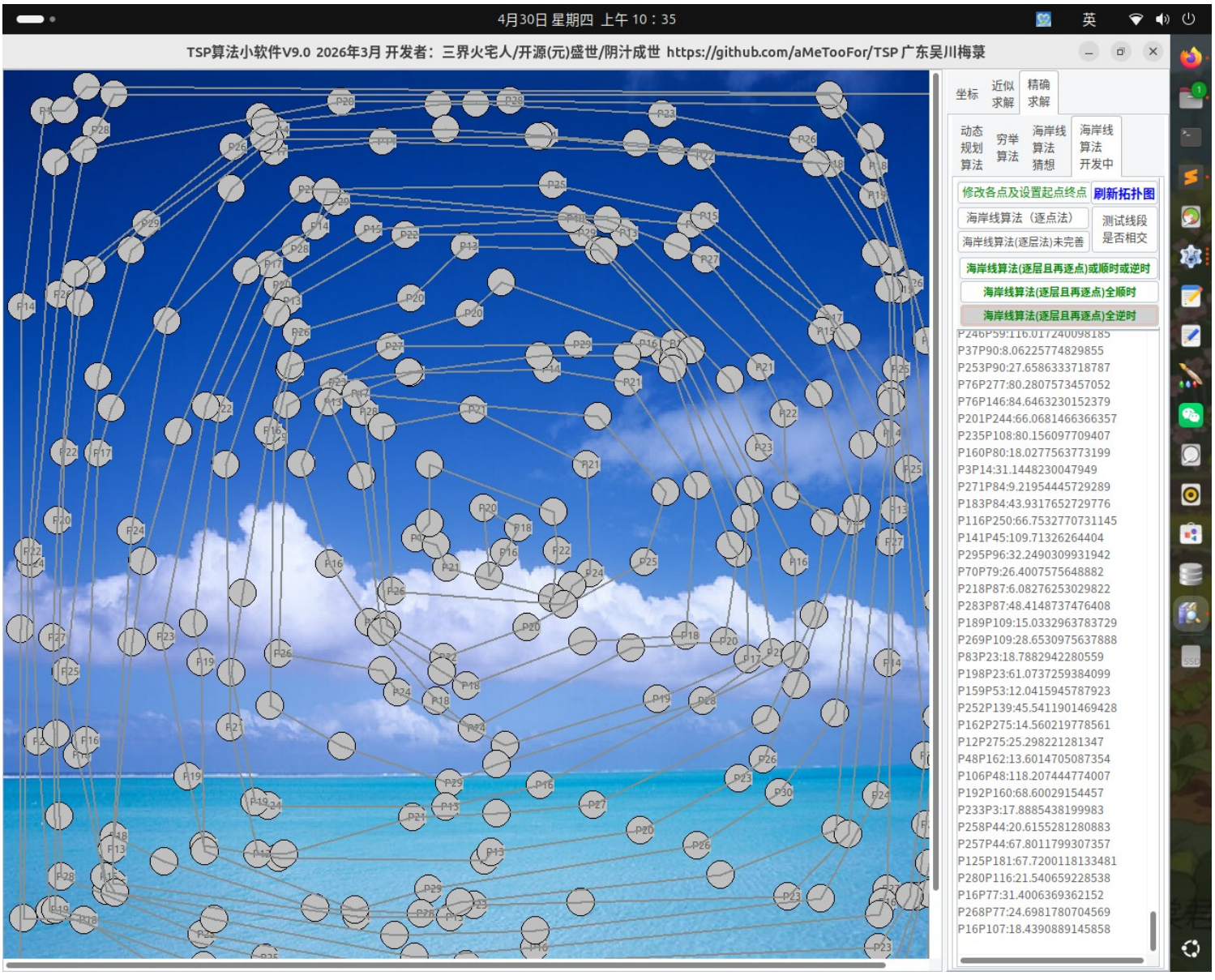
```
        begin
          curp := i;
        end;
      end
      else     //第一次
        curp := i;
    end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := 0;
    TPointLine(PointList[curp]).floor := 0;
  end;
end;
if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  floorLine.add(IntToStr(curp));
//memo_seaLine.Lines.Add('3:' + path);
application.ProcessMessages;
////////////////////////////////////
///########################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).py = TPointLine(PointList[maxy]).py) then
  begin
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px), 10) +
      '              ' + IntToStr(i));
    application.ProcessMessages;
  end;
end;
sortlist.sort;
//for i := 0 to sortlist.Count - 1 do
for i := sortlist.Count - 1 downto 0 do
  floorLine.add(trim(rightstr(sortlist[i], 10)));//四正之午正（含午初）
//  maxy := StrToInt(sortlist[0]);
// if floorLine.IndexOf(IntToStr(maxy)) <= -1 then
//   floorLine.add(IntToStr(maxy));
if sortlist.Count > 0 then
  maxy := StrToInt(trim(rightstr(sortlist[0], 10)));//午末
ss := floorLine.Text;
application.ProcessMessages;
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if (TPointLine(PointList[i]).px = TPointLine(PointList[minx]).px) then
  begin
    sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py), 10) +
      '              ' + IntToStr(i));
```

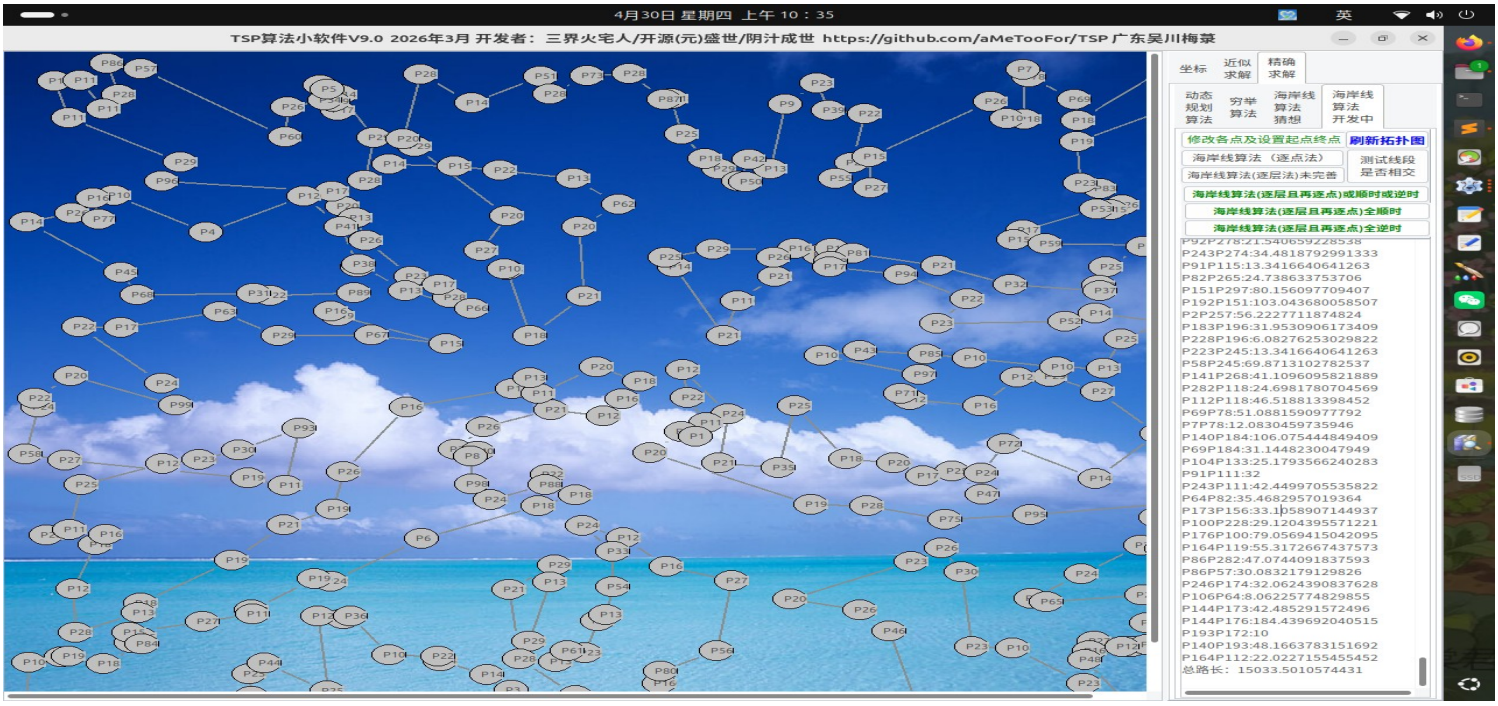
```
    application.ProcessMessages;
   end;
  end;
  sortlist.sort;
  if sortlist.Count > 0 then
   minx := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));//酉之初

  ss := floorLine.Text;
  application.ProcessMessages;
  ////////////////////////////////////
  application.ProcessMessages;
  curp := maxy;
  curp0 := -1;
  path := '';
  while curp0 <> curp do//午末与酉初之间的四隅之申
  begin
   application.ProcessMessages;
   curp0 := curp;
   //for i := 0 to PointList.Count - 1 do
   for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
     if (TPointLine(PointList[i]).px > TPointLine(PointList[curp0]).px) then
      continue;
     if (TPointLine(PointList[i]).py > TPointLine(PointList[curp0]).py) then
      continue;
     if (TPointLine(PointList[i]).py >= TPointLine(PointList[maxy]).py) then
      continue;
     if (TPointLine(PointList[i]).px > TPointLine(PointList[maxy]).px) then
      continue;
     if (TPointLine(PointList[i]).py < TPointLine(PointList[minx]).py) then
      continue;
     if (TPointLine(PointList[i]).px <= TPointLine(PointList[minx]).px) then
      continue;
     //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
     // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
     if (i = curp0) then
      continue;
     if (i = minx) then
      continue;
     if (i = maxx) then
      continue;
     if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
     begin       //k1,k2,a,b,c,d:double;
      a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
      b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
      c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
      d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
      //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
      //if ((TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py) /
      //  (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px)) <
```

```
      //  ((TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py) /
      //  (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px)) then
      if ((b = 0) and (d <> 0)) then
      begin
        continue;
      end;
      if ((b <> 0) and (d = 0)) then
      begin
        curp := i;
      end;
      if ((b = 0) and (d = 0)) then
      begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
        if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
          //这里本可以加等号，也可不要等号
          curp := i;
      end;
      if ((b <> 0) and (d <> 0)) then
        if ((a * d) = (b * c)) then
          //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        begin
          //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
          if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
            //这里本可以加等号，也可不要等号
            curp := i;
        end
        else if ((a * d) < (b * c)) then
          //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        begin
          curp := i;
        end;
     end
     else     //第一次
       curp := i;
   end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := 0;
    TPointLine(PointList[curp]).floor := 0;
  end;
end;
if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  floorLine.add(IntToStr(curp));
// memo_seaLine.Lines.Add('4:' + path);
application.ProcessMessages;
////////////////////////////////////
//sortlist.Clear;
//for i := 0 to PointList.Count - 1 do
```

```
//begin
//  if (TPointLine(PointList[i]).px = TPointLine(PointList[minx]).px) then
//  begin
//    sortlist.add(IntToStr(TPointLine(PointList[i]).py) + '               ' +
//      IntToStr(i));
//  end;
//end;
//sortlist.sort;
//for i := 0 to sortlist.Count - 1 do
//  // for i := sortlist.Count - 1 downto 0 do
//  floorLine.add(trim(rightstr(sortlist[i], 10)));
// minx := strtoint(trim(rightstr(sortlist[sortlist.Count - 1], 10)));
//if floorLine.IndexOf(IntToStr(minx)) <= 0 then
//  floorLine.add(IntToStr(minx));
//################
ss := floorLine.Text;
path := '';
for i := 0 to floorLine.Count - 1 do
begin
  TPointLine(PointList[StrToInt(floorLine[i mod floorLine.Count])]).floor := 0;
  pline := TlinePoint.Create(nil);
  pcode1 := pcodes[StrToInt(floorLine[i mod floorLine.Count])];
  pcode2 := pcodes[StrToInt(floorLine[(i + 1) mod floorLine.Count])];
  pcode1Shape := getpcode(pcode1);
  pcode2Shape := getpcode(pcode2);
  pcode1Pointer.x := pcode1Shape.Left + (pcode1Shape.Width div 2);
  pcode1Pointer.y := pcode1Shape.top + (pcode1Shape.Height div 2);
  pcode2Pointer.x := pcode2Shape.Left + (pcode2Shape.Width div 2);
  pcode2Pointer.y := pcode2Shape.top + (pcode2Shape.Height div 2);
  //self.Image1.Canvas.Line(pcode1Pointer,pcode2Pointer);

  if ResultPathEdgesRed.IndexOf(pcode1 + pcode2) >= 0 then
  begin
    ScrollBox1.Canvas.pen.color := clHighlight;//clred;
    ScrollBox1.Canvas.pen.Width := 4;
  end
  else
  begin
    ScrollBox1.Canvas.pen.color := clgrayText;
    ScrollBox1.Canvas.pen.Width := 2;
  end;
  self.ScrollBox1.Canvas.Line(pcode1Pointer, pcode2Pointer);  //ok
  application.ProcessMessages;
  pline.beginPcode := pcode1;
  pline.endPcode := pcode2;
  pline.beginPoint := pcode1Pointer;
  pline.endPoint := pcode2Pointer;
  pline.floor := 0;  ///想不到，后来修改时，少加这句，就难找不易
  linelist.Add(pline);
  if pos('-->' + pcode1, path) <= 0 then
    path := path + '-->' + pcode1;
```

```
    if pos('-->' + pcode2, path) <= 0 then
      path := path + '-->' + pcode2;
  end;
  memo_seaLine.Lines.Add(path);
  sortlist.Free;
  floorLine.Free;
  ////////////////////////////////////////////////////////////////
  //refreshFromLineList(nil);
end;
=============================================
procedure TMain.btn_seaLineFloorPointClick(Sender: TObject);
var
  i, j, k, jj, jjj, ij, ik, lmn: integer;      //memo_seaLine
  minx0, miny0, maxx0, maxy0: integer;//TPoint;
  //curp, curp0: integer;
  pline, ppline: TlinePoint;
  //pcode1, pcode2: string;
  //pcode1Shape, pcode2Shape: tshape;
  //pcode1Pointer, pcode2Pointer: tpoint;
```






```
  //pcode00: array of string;
  //pcode01: array of string;
  //pcode02: array of string;
  minH, curminH, onelong, alllong: double;
  cc: integer;
  ss: string;
  //ab, ac, bc, H, Sabc, minHH, minHHH, pp: real;
  //p, t, aaa, bbb, ccc, pp1, pp2: tpoint;
  //path, pathAlready: string;
  //p1, p2, p11, p22, oneline: string;
  //onelinelength, alllinelong: double;
  tempLineList: TFPList;//TPointerList;//TList;   // TlinePoint
  storeyLineList: array of TStringList;//TPointerList;//TList;   // TlinePoint
  floorSumCurminH: double;
  floorSumCurminHto: double;
  floorSumCurminHdownto: double;
////////////////////////////////////////////////////////
  function PPLong(a, b: Tpoint): double;
  begin
    Result := 0.0;
    Result := sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
  end;
  function wlog(w: string): integer;
  begin
    Result := 1;
    cc := citycount;
    //cc:=9;
    //cc:=9;
    if (ik > (cc - 2)) then
      //if (iii>(7)) then
      memo_seaLine.Lines.Add(w);
  end;
```

```
function sealineLayer(minx, miny, maxx, maxy: integer): integer;
var
  i, j, curTOPstorey: integer;      //memo_seaLine
  curp, curp0, tempnextp: integer;//TPoint;
  pline: TlinePoint;
  pcode1, pcode2: string;
  pcode1Shape, pcode2Shape: tshape;
  pcode1Pointer, pcode2Pointer: tpoint;
  path, ss: string;
  minpx, minpy, maxpx, maxpy: integer;
  sortlist, floorLine: TStringList;
  k1, k2, a, b, c, d: double;
begin
  sortlist := TStringList.Create;
  floorLine := TStringList.Create;
  sortlist.Clear;
  floorLine.Clear;
  ////////////////////////////////////     四正     子午卯酉
  //minpx,minpy,maxpx,maxpy:integer;
  minpx := TPointLine(PointList[minx]).px;
  minpy := TPointLine(PointList[miny]).py;
  maxpx := TPointLine(PointList[maxx]).px;
  maxpy := TPointLine(PointList[maxy]).py;
  ///#######################################
  sortlist.Clear;
  for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
      break;
  minx := i;//TPoint(PointList[0]);
  miny := minx;
  maxx := minx;
  maxy := minx;
  for i := 0 to PointList.Count - 1 do
    // PointList: TFPList;//TPointerList;//TList;  //TPointline
    if TPointLine(PointList[i]).floor = -1 then
    begin
      // ss:= TPointLine(PointList[i]).pcode;
      if TPointLine(PointList[i]).pcode = 'P9' then
        application.ProcessMessages;
      if (TPointLine(PointList[i]).px < TPointLine(PointList[minx]).px) then
        minx := i;//  TPoint(PointList[i]);
      if (TPointLine(PointList[i]).py < TPointLine(PointList[miny]).py) then
        miny := i;//  TPoint(PointList[i]);
      if (TPointLine(PointList[i]).px > TPointLine(PointList[maxx]).px) then
        maxx := i;//  TPoint(PointList[i]);
      if (TPointLine(PointList[i]).py > TPointLine(PointList[maxy]).py) then
        maxy := i;//  TPoint(PointList[i]);
    end;
  memo_seaLine.Lines.Add('第' + IntToStr(length(storeyLineList) + 1) +
    '层围城线围墙：');
  memo_seaLine.Lines.Add('minx=' + pcodes[minx]);
```

```
memo_seaLine.Lines.Add('miny=' + pcodes[miny]);
memo_seaLine.Lines.Add('maxx=' + pcodes[maxx]);
memo_seaLine.Lines.Add('maxy=' + pcodes[maxy]);
application.ProcessMessages;
//tempLineList:=TFPList.Create;//TPointerList;//TList;   // TlinePoint
curTOPstorey := length(storeyLineList);
setlength(storeyLineList, curTOPstorey + 1);
storeyLineList[curTOPstorey] := TStringList.Create;
////////////////////////////////////// 四正    子午卯酉
//minpx,minpy,maxpx,maxpy:integer;
minpx := TPointLine(PointList[minx]).px;
minpy := TPointLine(PointList[miny]).py;
maxpx := TPointLine(PointList[maxx]).px;
maxpy := TPointLine(PointList[maxy]).py;
///#######################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
 //for i := PointList.Count - 1 downto 0 do
begin
 if TPointLine(PointList[i]).floor = -1 then
  if (TPointLine(PointList[i]).px = TPointLine(PointList[minx]).px) then
  begin
   //TPointLine(PointList[i]).floor := curTOPstorey;
   //sortlist.add(inttostr(TPointLine(PointList[i]).py)+'              '+TPointLine(PointList[i]).pcode);
   sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py),
     10) + '              ' + IntToStr(i));
  end;
end;
sortlist.sort;
//for i := 0 to sortlist.Count - 1 do
for i := sortlist.Count - 1 downto 0 do
begin
 floorLine.add(trim(rightstr(sortlist[i], 10)));
 //四正：酉正，从酉初到酉末
 TPointLine(PointList[StrToInt(trim(rightstr(sortlist[i], 10)))]).floor :=
  curTOPstorey;
end;
ss := floorLine.Text;
if sortlist.Count > 0 then
 minx := StrToInt(trim(rightstr(sortlist[0], 10)));  //酉时末，顶上
///#######################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
 if TPointLine(PointList[i]).floor = -1 then
  if (TPointLine(PointList[i]).py = TPointLine(PointList[miny]).py) then
  begin
   sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px),
     10) + '              ' + IntToStr(i));
  end;
end;
```

```
sortlist.sort;
if sortlist.Count > 0 then
  //miny := StrToInt(trim(rightstr(sortlist[sortlist.Count-1], 10)));//子时初
  miny := StrToInt(trim(rightstr(sortlist[0], 10)));//子时初
application.ProcessMessages;
curp := minx;
curp0 := -1;
path := '';
while curp0 <> curp do//八极之四隅之亥，从酉末到子初也
begin
  //if curp<>-1 then
  application.ProcessMessages;
  curp0 := curp;
  for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
      if (TPointLine(PointList[i]).px < TPointLine(PointList[curp0]).px) then
        continue;
      if (TPointLine(PointList[i]).py > TPointLine(PointList[curp0]).py) then
        continue;
      if (TPointLine(PointList[i]).py > TPointLine(PointList[minx]).py) then
        continue;
      if (TPointLine(PointList[i]).px <= TPointLine(PointList[minx]).px) then
        continue;
      if (TPointLine(PointList[i]).py <= TPointLine(PointList[miny]).py) then
        continue;
      if (TPointLine(PointList[i]).px > TPointLine(PointList[miny]).px) then
        continue;
      //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
      // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
      if (i = curp0) then
        continue;
      if (i = miny) then
        continue;
      if (i = minx) then
        continue;
      if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
      begin        //k1,k2,a,b,c,d:double;
        a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
        b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
        c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
        d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
        //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        //if ((TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py) /
        //  (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px)) <
        //  ((TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py) /
        //  (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px)) then
        if ((b = 0) and (d <> 0)) then
        begin
          continue;
        end;
```

```
        if ((b <> 0) and (d = 0)) then
        begin
          curp := i;
        end;
        if ((b = 0) and (d = 0)) then
        begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
          if TPointLine(PointList[i]).py > TPointLine(PointList[curp]).py then
            //这里本可以加等号，也可不要等号
            curp := i;
        end;
        if ((b <> 0) and (d <> 0)) then
          if ((a * d) = (b * c)) then
            //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
          begin
            //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
            if TPointLine(PointList[i]).py > TPointLine(PointList[curp]).py then
              //这里本可以加等号，也可不要等号
              curp := i;
          end
          else if ((a * d) < (b * c)) then
            //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
          begin
            curp := i;
          end;
      end
      else     //第一次
        curp := i;
    end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := curTOPstorey;
    TPointLine(PointList[curp]).floor := curTOPstorey;
  end;
end;
if miny <> curp then
begin
  if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  begin
    floorLine.add(IntToStr(curp));
    TPointLine(pointlist[curp]).floor := curTOPstorey;
  end;
end;
//TPointLine(pointlist[miny]).floor := curTOPstorey;
//memo_seaLine.Lines.Add('1:' + path);
ss := floorLine.Text;
application.ProcessMessages;
////////////////////////////////////////////////////////////////
```

```
///#####################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if TPointLine(PointList[i]).floor = -1 then
    if (TPointLine(PointList[i]).py = TPointLine(PointList[miny]).py) then
    begin
      sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px),
        10) + '              ' + IntToStr(i));
    end;
end;
sortlist.sort;
for i := 0 to sortlist.Count - 1 do
  // for i := sortlist.Count - 1 downto 0 do
begin
  floorLine.add(trim(rightstr(sortlist[i], 10)));
  //四正：子正，从子初到子末
  TPointLine(PointList[StrToInt(trim(rightstr(sortlist[i], 10)))]).floor :=
    curTOPstorey;
end;
ss := sortlist.Text;
if sortlist.Count > 0 then
  miny := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));
//子时末，右侧
///#####################################
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if TPointLine(PointList[i]).floor = -1 then
    if (TPointLine(PointList[i]).px = TPointLine(PointList[maxx]).px) then
    begin
      sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py),
        10) + '              ' + IntToStr(i));
    end;
end;
sortlist.sort;
if sortlist.Count > 0 then
  // maxx := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));//卯时初
  maxx := StrToInt(trim(rightstr(sortlist[0], 10)));//卯时初
//////////////////////////////////////
curp := miny;
curp0 := -1;
path := '';
while curp0 <> curp do //八极之四隅之寅，从子末到卯初
begin
  application.ProcessMessages;
  curp0 := curp;
  for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
      if (TPointLine(PointList[i]).px < TPointLine(PointList[curp0]).px) then
```

```
  continue;
 if (TPointLine(PointList[i]).py < TPointLine(PointList[curp0]).py) then
  continue;
 if (TPointLine(PointList[i]).py > TPointLine(PointList[maxx]).py) then
  continue;
 if (TPointLine(PointList[i]).px >= TPointLine(PointList[maxx]).px) then
  continue;
 if (TPointLine(PointList[i]).py <= TPointLine(PointList[miny]).py) then
  continue;
 if (TPointLine(PointList[i]).px < TPointLine(PointList[miny]).px) then
  continue;
 //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
 // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
 if (i = curp0) then
  continue;
 if (i = miny) then
  continue;
 if (i = maxx) then
  continue;
 if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
 begin       //k1,k2,a,b,c,d:double;
  a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
  b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
  c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
  d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
  //  k1:=a/b ;  k2:=c/d;     k1<k2    a/b<c/d   ad<bc
  if ((b = 0) and (d <> 0)) then
  begin
   continue;
  end;
  if ((b <> 0) and (d = 0)) then
  begin
   curp := i;
  end;
  if ((b = 0) and (d = 0)) then
  begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
   if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
    //这里本可以加等号，也可不要等号
    curp := i;
  end;
  if ((b <> 0) and (d <> 0)) then
   if ((a * d) = (b * c)) then
    //  k1:=a/b ;  k2:=c/d;     k1<k2    a/b<c/d   ad<bc
   begin
    //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
    if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
     //这里本可以加等号，也可不要等号
     curp := i;
   end
   else if ((a * d) < (b * c)) then
    //  k1:=a/b ;  k2:=c/d;     k1<k2    a/b<c/d   ad<bc
```

```
        begin
          curp := i;
        end;
      end
      else     //第一次
        curp := i;
    end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := curTOPstorey;
    TPointLine(PointList[curp]).floor := curTOPstorey;
  end;
end;
if maxx <> curp then
begin
  if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  begin
    floorLine.add(IntToStr(curp));
    TPointLine(pointlist[curp]).floor := curTOPstorey;
  end;
end;
// memo_seaLine.Lines.Add('2:' + path);
ss := floorLine.Text;
application.ProcessMessages;
////////////////////////////////////////////////////////////////
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if TPointLine(PointList[i]).floor = -1 then
    if (TPointLine(PointList[i]).px = TPointLine(PointList[maxx]).px) then
    begin
      sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py),
        10) + '              ' + IntToStr(i));
    end;
end;
sortlist.sort;
for i := 0 to sortlist.Count - 1 do
  // for i := sortlist.Count - 1 downto 0 do
begin
  floorLine.add(trim(rightstr(sortlist[i], 10)));
  //四正：卯正，从卯初到卯末
  TPointLine(PointList[StrToInt(trim(rightstr(sortlist[i], 10)))]).floor :=
    curTOPstorey;
end;
if sortlist.Count > 0 then
  maxx := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));
//卯时末，右底
```

```
  sortlist.Clear;
  for i := 0 to PointList.Count - 1 do
  begin
   if TPointLine(PointList[i]).floor = -1 then
    if (TPointLine(PointList[i]).py = TPointLine(PointList[maxy]).py) then
    begin
     sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px),
       10) + '              ' + IntToStr(i));
    end;
  end;
  sortlist.sort;
  ss := sortlist.Text;
  if sortlist.Count > 0 then
   maxy := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));//午时初
  ///#######################################
  ss := floorLine.Text;
  application.ProcessMessages;
  curp := maxx;
  curp0 := -1;
  path := '';
  while curp0 <> curp do//八极之四隅之巳，从卯末到午初
  begin
   application.ProcessMessages;
   curp0 := curp;
   //for i := 0 to PointList.Count - 1 do
   for i := 0 to PointList.Count - 1 do
    if TPointLine(PointList[i]).floor = -1 then
    begin
     if (TPointLine(PointList[i]).px > TPointLine(PointList[curp0]).px) then
       continue;
     if (TPointLine(PointList[i]).py < TPointLine(PointList[curp0]).py) then
       continue;
     if (TPointLine(PointList[i]).py < TPointLine(PointList[maxx]).py) then
       continue;
     if (TPointLine(PointList[i]).px >= TPointLine(PointList[maxx]).px) then
       continue;
     if (TPointLine(PointList[i]).py >= TPointLine(PointList[maxy]).py) then
       continue;
     if (TPointLine(PointList[i]).px < TPointLine(PointList[maxy]).px) then
       continue;
     //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
     // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
     if (i = curp0) then
       continue;
     if (i = maxy) then
       continue;
     if (i = maxx) then
       continue;
     if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
     begin        //k1,k2,a,b,c,d:double;
       a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
```

```
      b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
      c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
      d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
      //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
      if ((b = 0) and (d <> 0)) then
      begin
        continue;
      end;
      if ((b <> 0) and (d = 0)) then
      begin
        curp := i;
      end;
      if ((b = 0) and (d = 0)) then
      begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
        if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
          //这里本可以加等号，也可不要等号
          curp := i;
      end;
      if ((b <> 0) and (d <> 0)) then
        if ((a * d) = (b * c)) then
          //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        begin
          //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
          if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
            //这里本可以加等号，也可不要等号
            curp := i;
        end
        else if ((a * d) < (b * c)) then
          //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
        begin
          curp := i;
        end;
      end
      else     //第一次
        curp := i;
    end;
  if curp0 <> curp then    //跳出循环后
  begin
    if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      floorLine.add(IntToStr(curp0));
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      floorLine.add(IntToStr(curp));
    TPointLine(PointList[curp0]).floor := curTOPstorey;
    TPointLine(PointList[curp]).floor := curTOPstorey;
  end;
end;
if maxy <> curp then
begin
  if floorLine.IndexOf(IntToStr(curp)) <= -1 then
  begin
    floorLine.add(IntToStr(curp));
```

```
    TPointLine(pointlist[curp]).floor := curTOPstorey;
  end;
end;
//  memo_seaLine.Lines.Add('3:' + path);
ss := floorLine.Text;
application.ProcessMessages;
////////////////////////////////////
sortlist.Clear;
for i := 0 to PointList.Count - 1 do
begin
  if TPointLine(PointList[i]).floor = -1 then
    if (TPointLine(PointList[i]).py = TPointLine(PointList[maxy]).py) then
    begin
      sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).px),
        10) + '              ' + IntToStr(i));
    end;
end;
sortlist.sort;
//for i := 0 to sortlist.Count - 1 do
for i := sortlist.Count - 1 downto 0 do
begin
  floorLine.add(trim(rightstr(sortlist[i], 10)));  //四正之午正（含午初）
  TPointLine(PointList[StrToInt(trim(rightstr(sortlist[i], 10)))]).floor :=
    curTOPstorey;
end;
if sortlist.Count > 0 then
  maxy := StrToInt(trim(rightstr(sortlist[0], 10)));//午末
ss := floorLine.Text;
application.ProcessMessages;
//sortlist.Clear;
//for i := 0 to PointList.Count - 1 do
//begin
//  if TPointLine(PointList[i]).floor = -1 then  //这里又产生 BUG，如果为 floor = -1，则没有数据，下标-1 报错
//    if (TPointLine(PointList[i]).px = TPointLine(PointList[minx]).px) then
//    begin
//      sortlist.add(rightstr('0000000000' + IntToStr(TPointLine(PointList[i]).py), 10) +
//        '              ' + IntToStr(i));
//    end;
//end;
//sortlist.sort;
// minx := StrToInt(trim(rightstr(sortlist[sortlist.Count - 1], 10)));//酉之初
minx := StrToInt(floorLine[0]);//酉之初
////////////////////////////////////
application.ProcessMessages;
curp := maxy;
curp0 := -1;
path := '';
// tempnextp := -1;
while curp0 <> curp do//午末与酉初之间的四隅之申
begin
  application.ProcessMessages;
```

```
curp0 := curp;
//for i := 0 to PointList.Count - 1 do
for i := 0 to PointList.Count - 1 do
 if TPointLine(PointList[i]).floor = -1 then
 begin
  if (TPointLine(PointList[i]).px > TPointLine(PointList[curp0]).px) then
   continue;
  if (TPointLine(PointList[i]).py > TPointLine(PointList[curp0]).py) then
   continue;
  if (TPointLine(PointList[i]).py >= TPointLine(PointList[maxy]).py) then
   continue;
  if (TPointLine(PointList[i]).px > TPointLine(PointList[maxy]).px) then
   continue;
  if (TPointLine(PointList[i]).py < TPointLine(PointList[minx]).py) then
   continue;
  if (TPointLine(PointList[i]).px <= TPointLine(PointList[minx]).px) then
   continue;
  //if (TPointLine(PointList[i]).py = TPointLine(PointList[maxx]).py) then
  // if (TPointLine(PointList[i]).px = TPointLine(PointList[miny]).px) then
  if (i = curp0) then
   continue;
  if (i = minx) then
   continue;
  if (i = maxx) then
   continue;
  if (curp0 <> curp) then  //  中间    curp0--curp   curp0--i
  begin       //k1,k2,a,b,c,d:double;
   a := (TPointLine(PointList[i]).py - TPointLine(PointList[curp0]).py);
   b := (TPointLine(PointList[i]).px - TPointLine(PointList[curp0]).px);
   c := (TPointLine(PointList[curp]).py - TPointLine(PointList[curp0]).py);
   d := (TPointLine(PointList[curp]).px - TPointLine(PointList[curp0]).px);
   //  k1:=a/b ;  k2:=c/d;     k1<k2    a/b<c/d   ad<bc
   if ((b = 0) and (d <> 0)) then
   begin
    continue;
   end;
   if ((b <> 0) and (d = 0)) then
   begin
    curp := i;
   end;
   if ((b = 0) and (d = 0)) then
   begin          //curp0,curp,i 同为垂直时，取垂直最近 curp0 者
    if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
     //这里本可以加等号，也可不要等号
     curp := i;
   end;
   if ((b <> 0) and (d <> 0)) then
    if ((a * d) = (b * c)) then
     //  k1:=a/b ;  k2:=c/d;     k1<k2    a/b<c/d   ad<bc
    begin
     //curp0,curp,i 同为同一条斜率直线上时，取最近 curp0 者
```

```
          if TPointLine(PointList[i]).py < TPointLine(PointList[curp]).py then
            //这里本可以加等号，也可不要等号
            curp := i;
          end
          else if ((a * d) < (b * c)) then
            //  k1:=a/b ;  k2:=c/d;      k1<k2    a/b<c/d   ad<bc
          begin
            curp := i;
          end;
        end
        else     //第一次
          curp := i;
      end;
    if curp0 <> curp then    //跳出循环后
    begin
      if floorLine.IndexOf(IntToStr(curp0)) <= -1 then
      begin
        floorLine.add(IntToStr(curp0));
        TPointLine(PointList[curp0]).floor := curTOPstorey;
      end;
      if floorLine.IndexOf(IntToStr(curp)) <= -1 then
      begin
        floorLine.add(IntToStr(curp));
        TPointLine(PointList[curp]).floor := curTOPstorey;
      end;
    end;
  end;
  if minx <> curp then
  begin
    if floorLine.IndexOf(IntToStr(curp)) <= -1 then
    begin
      floorLine.add(IntToStr(curp));
      TPointLine(pointlist[curp]).floor := curTOPstorey;
    end;
  end;
  // memo_seaLine.Lines.Add('4:' + path);
  application.ProcessMessages;
  ////////////////////////////////////
  TPointLine(pointlist[miny]).floor := curTOPstorey;
  TPointLine(pointlist[maxx]).floor := curTOPstorey;
  TPointLine(pointlist[maxy]).floor := curTOPstorey;
  TPointLine(pointlist[minx]).floor := curTOPstorey;
  if storeyLineList[curTOPstorey].Count = 0 then
    //这是仅剩下最后一点圆心的情况
  begin
    //ss := TPointLine(pointlist[minx]).Pcode;
    //storeyLineList[curTOPstorey].add(ss);
    //storeyLineList[curTOPstorey].add(ss);
  end;
  //for j := 0 to linelist.Count - 1 do
  //begin
```

```
//  if  storeyLineList[i].IndexOf(TlinePoint(linelist[j]).beginPcode)<=0 then
//  storeyLineList[i].add(TlinePoint(linelist[j]).beginPcode);
//  if  storeyLineList[i].IndexOf(TlinePoint(linelist[j]).endPcode)<=0 then
//  storeyLineList[i].add(TlinePoint(linelist[j]).endPcode);
//end;
////////////////////////////////////////////////////////////////
//refreshFromLineList(nil);
ss := floorLine.Text;
path := '';
//if curTOPstorey>=2 then
// exit;
for i := 0 to floorLine.Count - 1 do
begin
 pline := TlinePoint.Create(nil);
 pcode1 := pcodes[StrToInt(floorLine[i mod floorLine.Count])];
 pcode2 := pcodes[StrToInt(floorLine[(i + 1) mod floorLine.Count])];
 pcode1Shape := getpcode(pcode1);
 pcode2Shape := getpcode(pcode2);
 pcode1Pointer.x := pcode1Shape.Left + (pcode1Shape.Width div 2);
 pcode1Pointer.y := pcode1Shape.top + (pcode1Shape.Height div 2);
 pcode2Pointer.x := pcode2Shape.Left + (pcode2Shape.Width div 2);
 pcode2Pointer.y := pcode2Shape.top + (pcode2Shape.Height div 2);
 //self.Image1.Canvas.Line(pcode1Pointer,pcode2Pointer);
 if ResultPathEdgesRed.IndexOf(pcode1 + pcode2) >= 0 then
 begin
  ScrollBox1.Canvas.pen.color := clHighlight;//clred;
  ScrollBox1.Canvas.pen.Width := 4;
 end
 else
 begin
  ScrollBox1.Canvas.pen.color := clgrayText;
  ScrollBox1.Canvas.pen.Width := 2;
 end;
 self.ScrollBox1.Canvas.Line(pcode1Pointer, pcode2Pointer);  //ok
 pline.beginPcode := pcode1;
 pline.endPcode := pcode2;
 pline.beginPoint := pcode1Pointer;
 pline.endPoint := pcode2Pointer;
 pline.floor := curTOPstorey;
 ///想不到，后来修改时，少加这句，就难找不易
 linelist.Add(pline);
 if pos('-->' + pcode1, path) <= 0 then
  path := path + '-->' + pcode1;
 if pos('-->' + pcode2, path) <= 0 then
  path := path + '-->' + pcode2;
 TPointLine(pointlist[StrToInt(floorLine[i mod floorLine.Count])]).floor :=
  curTOPstorey;
 if storeyLineList[curTOPstorey].IndexOf(
  TPointLine(pointlist[StrToInt(floorLine[i mod floorLine.Count])]).Pcode) <=
  -1 then
  storeyLineList[curTOPstorey].add(
```

```
        TPointLine(pointlist[StrToInt(floorLine[i mod floorLine.Count])]).Pcode);
    end;
    storeyLineList[curTOPstorey].add(
      TPointLine(pointlist[StrToInt(floorLine[0])]).Pcode);
    memo_seaLine.Lines.Add(path);
    sortlist.Free;
    floorLine.Free;
    ////////////////////////////////////////////////////////////////
    //refreshFromLineList(nil);
  end;
begin
  memo_seaLine.Lines.Clear;
  pcodeAndPointLineListFromSQL;
  //for i := 1 to PointList.Count - 2 do
  //  for j := i + 1 to PointList.Count - 1 do
  //    if TPointLine(PointList[i]).px = TPointLine(PointList[j]).px then
  //      if TPointLine(PointList[i]).py = TPointLine(PointList[j]).py then
  //      begin
  //        ss := TPointLine(PointList[i]).Pcode + '坐标相同' +
  //          TPointLine(PointList[j]).Pcode + '暂将' +
  //          TPointLine(PointList[j]).Pcode +
  //          '的 x 与 y 坐标像素值各加 1 像素临时处理';
  //        ShowMessage(ss);    //P229
  //         // TPointLine(PointList[j]).px := TPointLine(PointList[j]).px + 1;
  //          TPointLine(PointList[j]).py := TPointLine(PointList[j]).py + 1;
  //      end;
  for i := 1 to PointList.Count - 1 do
    TPointLine(PointList[i]).floor := -1;
  for i := 1 to lineList.Count - 1 do
    TLinePoint(lineList[i]).floor := -1;
  minx0 := 0;//TPoint(PointList[0]);
  miny0 := minx0;
  maxx0 := minx0;
  maxy0 := minx0;
  for i := 1 to PointList.Count - 1 do
    // PointList: TFPList;//TPointerList;//TList;  //TPointline
  begin
    if (TPointLine(PointList[i]).px < TPointLine(PointList[minx0]).px) then
      minx0 := i;//  TPoint(PointList[i]);
    if (TPointLine(PointList[i]).py < TPointLine(PointList[miny0]).py) then
      miny0 := i;//  TPoint(PointList[i]);
    if (TPointLine(PointList[i]).px > TPointLine(PointList[maxx0]).px) then
      maxx0 := i;//  TPoint(PointList[i]);
    if (TPointLine(PointList[i]).py > TPointLine(PointList[maxy0]).py) then
      maxy0 := i;//  TPoint(PointList[i]);
  end;
  memo_seaLine.Lines.Add('minx=' + pcodes[minx0]);
  memo_seaLine.Lines.Add('miny=' + pcodes[miny0]);
  memo_seaLine.Lines.Add('maxx=' + pcodes[maxx0]);
  memo_seaLine.Lines.Add('maxy=' + pcodes[maxy0]);
  application.ProcessMessages;
```

```
memo_seaLine.Lines.Add('最外层围城线围墙：');
for i := lineList.Count - 1 downto 0 do
begin
 try
  //if tobject(shapeList.Items[i]) is TlinePoint then
  if lineList.Items[i] <> nil then
  begin
   TlinePoint(lineList.Items[i]).Free;
   lineList.Items[i] := nil;
   lineList.Count := lineList.Count - 1;
   application.ProcessMessages;
  end;
 except
 end;
 application.ProcessMessages;
end;
lineList.Clear;
for i := TPsubpPointList.Count - 1 downto 0 do
begin
 try
  if (TPsubpPointList.Items[i]) <> nil then
  begin
   TPsubpPoint(TPsubpPointList.Items[i]).Free;
   TPsubpPointList.Items[i] := nil;
   TPsubpPointList.Count := TPsubpPointList.Count - 1;
   application.ProcessMessages;
  end;
 except
 end;
 application.ProcessMessages;
end;
TPsubpPointList.Clear;
image1.Refresh;
////////////////////////////////////////////////////////////////
sealine000(minx0, miny0, maxx0, maxy0);       //最外层，另外计算
//exit;
////////////////////////////////////////////////
setlength(storeyLineList, 1);
storeyLineList[0] := TStringList.Create;
for i := 0 to linelist.Count - 1 do
begin
 if storeyLineList[0].IndexOf(TlinePoint(linelist[i]).beginPcode) <= -1 then
  storeyLineList[0].add(TlinePoint(linelist[i]).beginPcode);
 if storeyLineList[0].IndexOf(TlinePoint(linelist[i]).endPcode) <= -1 then
  storeyLineList[0].add(TlinePoint(linelist[i]).endPcode);
 TlinePoint(linelist[i]).floor := 0;
 // TPointLine(pointlist[getPcodeIndex(TlinePoint(linelist[i]).beginPcode)]).floor := 0 ;
 // TPointLine(pointlist[getPcodeIndex(TlinePoint(linelist[i]).endPcode)]).floor := 0 ;
end;
for i := 0 to storeyLineList[0].Count - 1 do
begin
```

```
    ss := (storeyLineList[0][i]);
    j := getPcodeIndex(ss);
    TPointLine(pointlist[j]).floor := 0;
  end;
  TPointLine(pointlist[miny0]).floor := 0;
  TPointLine(pointlist[maxx0]).floor := 0;
  TPointLine(pointlist[maxy0]).floor := 0;
  TPointLine(pointlist[minx0]).floor := 0;
  ////////////////////////
  i := 1;
  while i > 0 do
  begin
    i := 0;
    for j := 0 to pointlist.Count - 1 do
      //pointlist 所有点集，未分层的全点集
      if TPointLine(pointlist[j]).floor = -1 then
        I := i + 1;
    if i = 1 then
      application.ProcessMessages;
    if i > 0 then
      seaLineLayer(0, 0, 0, 0);    //求出所有层
    //seaLineLayer000(0, 0, 0, 0);    //求出所有层
    //sealineLayerTEST(0, 0, 0, 0);    //求出所有层
  end;
  ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
  //exit;
  for i := 0 to length(storeyLineList) - 1 do  //每一层点集
    memo_seaLine.Lines.Add(IntToStr(i) + '=' + storeyLineList[i].Text);
  //  exit;
  for i := 0 to length(storeyLineList) - 2 do  //每一层点集
    //for i := 0 to length(storeyLineList) - 3 do  //每一层点集 for test
  begin
    // for j := 0 to pointList.Count - 1 do   //pointlist 所有点集，未分层的全点集
    //正是这种循环不行，要用层内的 for j := 0 to storeyLineList[i].Count - 1 do
    //  for j := 0 to storeyLineList[i].Count - 1 do
    //  begin
    //    jjj := -1;
    //  for k := 0 to LineList.Count - 1 do     //pointlist 所有点集，未分层的全点集
    //正是这种循环不行，要用层内的 for k := 0 to storeyLineList[i+1].Count - 1 do
    // lmn := storeyLineList[i + 1].Count;
    //  ss:= storeyLineList[i].text;
    //  ss:= storeyLineList[i + 1].text;

    ////////////////////////////////////////////
    if TButton(Sender).Caption = '海岸线算法(逐层且再逐点)或顺时或逆时'
    then
      // if 1 = 2 then
    begin
      floorSumCurminH := 0;
      for k := 0 to storeyLineList[i + 1].Count - 2 do
```

```
 // j  out lower  k 要高层内层，j 在低层外层
 //for k :=  storeyLineList[i+1].Count - 2 downto 0 do     //  k in   high  k 要高层内层，j 在低层外层
begin
 for lmn := pointList.Count - 1 downto 0 do
  if TPointLine(pointList[lmn]).pcode = ((storeyLineList[i + 1])[k]) then
    break;
 if TPointLine(pointList[lmn]).floor = -111 then
  continue;
 minH := 10000000;
 ij := -1;
 ik := -1;
 jjj := -1;
 for j := 0 to storeyLineList[i].Count - 2 do
  // j  out lower  k 要高层内层，j 在低层外层
  //for j :=  storeyLineList[i].Count - 2 downto 0 do          //  k in   high  k 要高层内层，j 在低层外层
 begin
  curminH := (PPLong(fromPcodeGetPoint((storeyLineList[i + 1])
    [k]), fromPcodeGetPoint((storeyLineList[i])[j])) +
    PPLong(fromPcodeGetPoint((storeyLineList[i + 1])[k]),
    fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])) - PPLong(fromPcodeGetPoint(
    (storeyLineList[i])[j]), fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])));
  if curminH <= minH then
    //if curminH < minH then
    //此处加上等号，在三种坐标中，在坐标点是十分规则的矩形分布坐标轴一样整点时，也比原先的解优胜了
  begin   //此处未处理是不是相交线，所以可能时或偶有相交线出现乎
    minH := curminH;
    ij := j;    //ii  j    in   high
    ik := k;   //iii   k  out  lower
    jjj := 1;
  end;
 end;  //for j
 ////////////////////////////////////////
 if jjj = 1 then
 begin
  floorSumCurminH := floorSumCurminH + minH;
  for jjj := pointList.Count - 1 downto 0 do
    //将原先内层高层的点，先标记，到时全删除乎
    if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
      TPointLine(pointList[jjj]).floor := -111;
 end;
end;  //for k
floorSumCurminHto := floorSumCurminH;
for jjj := pointList.Count - 1 downto 0 do
 for ik := (storeyLineList[i + 1]).Count - 1 downto 0 do
  if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
    TPointLine(pointList[jjj]).floor := i + 1;
floorSumCurminH := 0;
//for k := 0 to storeyLineList[i + 1].Count - 2 do       // j  out lower  k 要高层内层，j 在低层外层
for k := storeyLineList[i + 1].Count - 2 downto 0 do
```

```
  //  k in   high  k 要高层内层，j 在低层外层
 begin
  for lmn := pointList.Count - 1 downto 0 do
   if TPointLine(pointList[lmn]).pcode = ((storeyLineList[i + 1])[k]) then
    break;
  if TPointLine(pointList[lmn]).floor = -111 then
   continue;
  minH := 10000000;
  ij := -1;
  ik := -1;
  jjj := -1;
  for j := 0 to storeyLineList[i].Count - 2 do
   // j  out lower  k 要高层内层，j 在低层外层
   //for j :=  storeyLineList[i].Count - 2 downto 0 do          //  k in   high  k 要高层内层，j 在低层外层
  begin
   curminH := (PPLong(fromPcodeGetPoint((storeyLineList[i + 1])
    [k]), fromPcodeGetPoint((storeyLineList[i])[j])) +
    PPLong(fromPcodeGetPoint((storeyLineList[i + 1])[k]),
    fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])) - PPLong(fromPcodeGetPoint(
    (storeyLineList[i])[j]), fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])));
   if curminH <= minH then
    //if curminH < minH then
    //此处加上等号，在三种坐标中，在坐标点是十分规则的矩形分布坐标轴一样整点时，也比原先的解优胜了
   begin   //此处未处理是不是相交线，所以可能时或偶有相交线出现乎
    minH := curminH;
    ij := j;    //ii  j    in   high
    ik := k;   //iii   k  out  lower
    jjj := 1;
   end;
  end;  //for j
  ////////////////////////////////////////
  if jjj = 1 then
  begin
   floorSumCurminH := floorSumCurminH + minH;
   for jjj := pointList.Count - 1 downto 0 do
    //将原先内层高层的点，先标记，到时全删除乎
    if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
     TPointLine(pointList[jjj]).floor := -111;
  end;
 end;  //for k
 floorSumCurminHdownto := floorSumCurminH;
 for jjj := pointList.Count - 1 downto 0 do
  for ik := (storeyLineList[i + 1]).Count - 1 downto 0 do
   if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
    TPointLine(pointList[jjj]).floor := i + 1;
 if floorSumCurminHto < floorSumCurminHdownto then
 begin
  for k := 0 to storeyLineList[i + 1].Count - 2 do
   // j  out lower  k 要高层内层，j 在低层外层
```

```
  //for k := storeyLineList[i + 1].Count - 2 downto 0 do      // k in   high k 要高层内层，j 在低层外层
  //  k in   high  k 要高层内层，j 在低层外层
  // if (TlinePoint(linelist[j]).floor = (i)) then
  //   if (TlinePoint(linelist[k]).floor = (i+1)) then
  //if (TlinePoint(linelist[k]).floor <>-111) then
begin
  for lmn := pointList.Count - 1 downto 0 do
    if TPointLine(pointList[lmn]).pcode = ((storeyLineList[i + 1])[k]) then
      break;
  if TPointLine(pointList[lmn]).floor = -111 then
    continue;
  minH := 10000000;
  ij := -1;
  ik := -1;
  jjj := -1;
  for j := 0 to storeyLineList[i].Count - 2 do
    // j  out lower  k 要高层内层，j 在低层外层
    //for j :=  storeyLineList[i].Count - 2 downto 0 do          //  k in   high  k 要高层内层，j 在低层外层
  begin
    curminH := (PPLong(fromPcodeGetPoint((storeyLineList[i + 1])
      [k]), fromPcodeGetPoint((storeyLineList[i])[j])) +
      PPLong(fromPcodeGetPoint((storeyLineList[i + 1])[k]),
      fromPcodeGetPoint((storeyLineList[i])
      [(j + 1) mod storeyLineList[i].Count])) -
      PPLong(fromPcodeGetPoint((storeyLineList[i])[j]),
      fromPcodeGetPoint((storeyLineList[i])
      [(j + 1) mod storeyLineList[i].Count])));
    if curminH <= minH then
      //if curminH < minH then
      //此处加上等号，在三种坐标中，在坐标点是十分规则的矩形分布坐标轴一样整点时，也比原先的解优胜了
    begin   //此处未处理是不是相交线，所以可能时或偶有相交线出现乎
      minH := curminH;
      ij := j;    //ii  j    in   high
      ik := k;   //iii   k  out  lower
      jjj := 1;
    end;
  end;  //for j
  ////////////////////////////////////////
  if jjj = 1 then
  begin
    pline := TlinePoint.Create(nil);
    pline.beginPcode := ((storeyLineList[i])[ij]);
    pline.endPcode := ((storeyLineList[i + 1])[ik]);
    pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])[ij]);
    pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
    pline.floor := i;
    linelist.Add(pline);
    pline := TlinePoint.Create(nil);
    pline.beginPcode := ((storeyLineList[i])
      [(ij + 1) mod storeyLineList[i].Count]);
    pline.endPcode := ((storeyLineList[i + 1])[ik]);
```

```
      pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])
        [(ij + 1) mod storeyLineList[i].Count]);
      pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
      pline.floor := i;
      linelist.Add(pline);
      // 三角形中两边替换一边，一种类似最短路径算法中的弗洛伊德算法中的松弛操作 relax operation
      //memo_seaLine.Lines.Add(((storeyLineList[i])[ij]) +
      //  ((storeyLineList[i + 1])[ik]) + '和' + ((storeyLineList[i])
      //  [(ij + 1) mod storeyLineList[i].Count]) + ((storeyLineList[i + 1])
      //  [ik]) + '替换' + ((storeyLineList[i])[ij]) + ((storeyLineList[i])
      //  [(ij + 1) mod storeyLineList[i].Count]));
      for jjj := lineList.Count - 1 downto 0 do
      begin
        //三角形中两边替换一边，将另一边先标记，到时统一删除，系统功能限制，不能立即删除乎
        if (TlinePoint(linelist[jjj]).beginPcode + TlinePoint(
          linelist[jjj]).endPcode) = ((storeyLineList[i])
          [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
          TlinePoint(linelist[jjj]).floor := -111;
        if (TlinePoint(linelist[jjj]).endPcode + TlinePoint(
          linelist[jjj]).beginPcode) = ((storeyLineList[i])
          [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
          TlinePoint(linelist[jjj]).floor := -111;
      end;
      for jjj := pointList.Count - 1 downto 0 do
        //将原先内层高层的点，先标记，到时全删除乎
        if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
          TPointLine(pointList[jjj]).floor := -111;
      //tstringlist(storeyLineList[i])[ij+1].Delete;
      //tstringlist(storeyLineList[i])[ij].Delete;
      TStringList(storeyLineList[i]).insert(ij + 1,
        TStringList(storeyLineList[i + 1])[ik]);
      //三角形中两边替换一边，实际相当于在原来的两点路径中插入一个新点
    end;
    //if jjj = 2 then
    begin
    end;
  end;  //for k
end;
if floorSumCurminHdownto <= floorSumCurminHto then
begin
  //     for k := 0 to storeyLineList[i + 1].Count - 2 do       // j  out lower  k 要高层内层，j 在低层外层
  for k := storeyLineList[i + 1].Count - 2 downto 0 do
    // k in   high k 要高层内层，j 在低层外层
    //  k in   high  k 要高层内层，j 在低层外层
    // if (TlinePoint(linelist[j]).floor = (i)) then
    //   if (TlinePoint(linelist[k]).floor = (i+1)) then
    //if (TlinePoint(linelist[k]).floor <>-111) then
  begin
    for lmn := pointList.Count - 1 downto 0 do
      if TPointLine(pointList[lmn]).pcode = ((storeyLineList[i + 1])[k]) then
        break;
```

```
if TPointLine(pointList[lmn]).floor = -111 then
  continue;
minH := 10000000;
ij := -1;
ik := -1;
jjj := -1;
for j := 0 to storeyLineList[i].Count - 2 do
  // j  out lower  k 要高层内层，j 在低层外层
  //for j :=  storeyLineList[i].Count - 2 downto 0 do          //  k in   high  k 要高层内层，j 在低层外层
begin
  curminH := (PPLong(fromPcodeGetPoint((storeyLineList[i + 1])
    [k]), fromPcodeGetPoint((storeyLineList[i])[j])) +
    PPLong(fromPcodeGetPoint((storeyLineList[i + 1])[k]),
    fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])) -
    PPLong(fromPcodeGetPoint((storeyLineList[i])[j]),
    fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])));
  if curminH <= minH then
    //if curminH < minH then
    //此处加上等号，在三种坐标中，在坐标点是十分规则的矩形分布坐标轴一样整点时，也比原先的解优胜了
  begin   //此处未处理是不是相交线，所以可能时或偶有相交线出现乎
    minH := curminH;
    ij := j;    //ii  j    in   high
    ik := k;   //iii   k  out  lower
    jjj := 1;
  end;
end;  //for j
////////////////////////////////////////
if jjj = 1 then
begin
  pline := TlinePoint.Create(nil);
  pline.beginPcode := ((storeyLineList[i])[ij]);
  pline.endPcode := ((storeyLineList[i + 1])[ik]);
  pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])[ij]);
  pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
  pline.floor := i;
  linelist.Add(pline);
  pline := TlinePoint.Create(nil);
  pline.beginPcode := ((storeyLineList[i])
    [(ij + 1) mod storeyLineList[i].Count]);
  pline.endPcode := ((storeyLineList[i + 1])[ik]);
  pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])
    [(ij + 1) mod storeyLineList[i].Count]);
  pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
  pline.floor := i;
  linelist.Add(pline);
  // 三角形中两边替换一边，一种类似最短路径算法中的弗洛伊德算法中的松弛操作 relax operation
  //memo_seaLine.Lines.Add(((storeyLineList[i])[ij]) +
  //  ((storeyLineList[i + 1])[ik]) + '和' + ((storeyLineList[i])
  //  [(ij + 1) mod storeyLineList[i].Count]) + ((storeyLineList[i + 1])
```

```
      //  [ik]) + '替换' + ((storeyLineList[i])[ij]) + ((storeyLineList[i])
      //  [(ij + 1) mod storeyLineList[i].Count]));
      for jjj := lineList.Count - 1 downto 0 do
      begin
       //三角形中两边替换一边，将另一边先标记，到时统一删除，系统功能限制，不能立即删除乎
       if (TlinePoint(linelist[jjj]).beginPcode + TlinePoint(
         linelist[jjj]).endPcode) = ((storeyLineList[i])
         [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
         TlinePoint(linelist[jjj]).floor := -111;
       if (TlinePoint(linelist[jjj]).endPcode + TlinePoint(
         linelist[jjj]).beginPcode) = ((storeyLineList[i])
         [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
         TlinePoint(linelist[jjj]).floor := -111;
      end;
      for jjj := pointList.Count - 1 downto 0 do
       //将原先内层高层的点，先标记，到时全删除乎
       if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
         TPointLine(pointList[jjj]).floor := -111;
      //tstringlist(storeyLineList[i])[ij+1].Delete;
      //tstringlist(storeyLineList[i])[ij].Delete;
      TStringList(storeyLineList[i]).insert(ij + 1,
       TStringList(storeyLineList[i + 1])[ik]);
      //三角形中两边替换一边，实际相当于在原来的两点路径中插入一个新点
     end;
     //if jjj = 2 then
     begin
     end;
   end;  //for k
  end;
 end;
 ////////////////////////////////////////////
 if TButton(Sender).Caption = '海岸线算法(逐层且再逐点)全顺时' then
  //if 1 = 2 then
  for k := 0 to storeyLineList[i + 1].Count - 2 do
   // j  out lower  k 要高层内层，j 在低层外层
   // for k := storeyLineList[i + 1].Count - 2 downto 0 do
   // k in   high k 要高层内层，j 在低层外层
   //  k in   high  k 要高层内层，j 在低层外层
   // if (TlinePoint(linelist[j]).floor = (i)) then
   //   if (TlinePoint(linelist[k]).floor = (i+1)) then
   //if (TlinePoint(linelist[k]).floor <>-111) then
  begin
   for lmn := pointList.Count - 1 downto 0 do
    if TPointLine(pointList[lmn]).pcode = ((storeyLineList[i + 1])[k]) then
      break;
   if TPointLine(pointList[lmn]).floor = -111 then
    continue;
   minH := 10000000;
   ij := -1;
   ik := -1;
   jjj := -1;
```

```
for j := 0 to storeyLineList[i].Count - 2 do
 // j  out lower  k 要高层内层，j 在低层外层
 //for j :=  storeyLineList[i].Count - 2 downto 0 do          //  k in   high  k 要高层内层，j 在低层外层
begin
 curminH := (PPLong(fromPcodeGetPoint((storeyLineList[i + 1])
  [k]), fromPcodeGetPoint((storeyLineList[i])[j])) +
  PPLong(fromPcodeGetPoint((storeyLineList[i + 1])[k]),
  fromPcodeGetPoint((storeyLineList[i])
  [(j + 1) mod storeyLineList[i].Count])) -
  PPLong(fromPcodeGetPoint((storeyLineList[i])[j]),
  fromPcodeGetPoint((storeyLineList[i])
  [(j + 1) mod storeyLineList[i].Count])));
 if curminH <= minH then
  //if curminH < minH then
  //此处加上等号，在三种坐标中，在坐标点是十分规则的矩形分布坐标轴一样整点时，也比原先的解优胜了
 begin   //此处未处理是不是相交线，所以可能时或偶有相交线出现乎
  minH := curminH;
  ij := j;   //ii  j    in   high
  ik := k;   //iii   k  out  lower
  jjj := 1;
 end;
end;  //for j
////////////////////////////////////////
if jjj = 1 then
begin
 pline := TlinePoint.Create(nil);
 pline.beginPcode := ((storeyLineList[i])[ij]);
 pline.endPcode := ((storeyLineList[i + 1])[ik]);
 pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])[ij]);
 pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
 pline.floor := i;
 linelist.Add(pline);
 pline := TlinePoint.Create(nil);
 pline.beginPcode := ((storeyLineList[i])
  [(ij + 1) mod storeyLineList[i].Count]);
 pline.endPcode := ((storeyLineList[i + 1])[ik]);
 pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])
  [(ij + 1) mod storeyLineList[i].Count]);
 pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
 pline.floor := i;
 linelist.Add(pline);
 // 三角形中两边替换一边，一种类似最短路径算法中的弗洛伊德算法中的松弛操作 relax operation
 //memo_seaLine.Lines.Add(((storeyLineList[i])[ij]) +
 //  ((storeyLineList[i + 1])[ik]) + '和' + ((storeyLineList[i])
 //  [(ij + 1) mod storeyLineList[i].Count]) + ((storeyLineList[i + 1])
 //  [ik]) + '替换' + ((storeyLineList[i])[ij]) + ((storeyLineList[i])
 //  [(ij + 1) mod storeyLineList[i].Count]));
 for jjj := lineList.Count - 1 downto 0 do
 begin
  //三角形中两边替换一边，将另一边先标记，到时统一删除，系统功能限制，不能立即删除乎
  if (TlinePoint(linelist[jjj]).beginPcode + TlinePoint(
```

```
          linelist[jjj]).endPcode) = ((storeyLineList[i])
          [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
          TlinePoint(linelist[jjj]).floor := -111;
        if (TlinePoint(linelist[jjj]).endPcode + TlinePoint(
          linelist[jjj]).beginPcode) = ((storeyLineList[i])
          [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
          TlinePoint(linelist[jjj]).floor := -111;
      end;
      for jjj := pointList.Count - 1 downto 0 do
       //将原先内层高层的点，先标记，到时全删除乎
       if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
         TPointLine(pointList[jjj]).floor := -111;
      //tstringlist(storeyLineList[i])[ij+1].Delete;
      //tstringlist(storeyLineList[i])[ij].Delete;
      TStringList(storeyLineList[i]).insert(ij + 1,
       TStringList(storeyLineList[i + 1])[ik]);
      //三角形中两边替换一边，实际相当于在原来的两点路径中插入一个新点
    end;
    //if jjj = 2 then
    begin
    end;
  end;  //for k
 if TButton(Sender).Caption = '海岸线算法(逐层且再逐点)全逆时' then
  //if 1 = 2 then
  //for k := 0 to storeyLineList[i + 1].Count - 2 do       // j  out lower  k 要高层内层，j 在低层外层
  for k := storeyLineList[i + 1].Count - 2 downto 0 do
    // k in   high k 要高层内层，j 在低层外层
    //  k in   high  k 要高层内层，j 在低层外层
    // if (TlinePoint(linelist[j]).floor = (i)) then
    //   if (TlinePoint(linelist[k]).floor = (i+1)) then
    //if (TlinePoint(linelist[k]).floor <>-111) then
   begin
    for lmn := pointList.Count - 1 downto 0 do
     if TPointLine(pointList[lmn]).pcode = ((storeyLineList[i + 1])[k]) then
       break;
    if TPointLine(pointList[lmn]).floor = -111 then
     continue;
    minH := 10000000;
    ij := -1;
    ik := -1;
    jjj := -1;
    for j := 0 to storeyLineList[i].Count - 2 do
     // j  out lower  k 要高层内层，j 在低层外层
     //for j :=  storeyLineList[i].Count - 2 downto 0 do          //  k in   high  k 要高层内层，j 在低层外层
    begin
     curminH := (PPLong(fromPcodeGetPoint((storeyLineList[i + 1])
       [k]), fromPcodeGetPoint((storeyLineList[i])[j])) +
       PPLong(fromPcodeGetPoint((storeyLineList[i + 1])[k]),
       fromPcodeGetPoint((storeyLineList[i])
       [(j + 1) mod storeyLineList[i].Count])) -
       PPLong(fromPcodeGetPoint((storeyLineList[i])[j]),
```

```
    fromPcodeGetPoint((storeyLineList[i])
    [(j + 1) mod storeyLineList[i].Count])));
   if curminH <= minH then
    //if curminH < minH then
    //此处加上等号，在三种坐标中，在坐标点是十分规则的矩形分布坐标轴一样整点时，也比原先的解优胜了
   begin   //此处未处理是不是相交线，所以可能时或偶有相交线出现乎
    minH := curminH;
    ij := j;    //ii  j    in   high
    ik := k;   //iii   k  out  lower
    jjj := 1;
   end;
  end;  //for j
//////////////////////////////////////
  if jjj = 1 then
  begin
   pline := TlinePoint.Create(nil);
   pline.beginPcode := ((storeyLineList[i])[ij]);
   pline.endPcode := ((storeyLineList[i + 1])[ik]);
   pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])[ij]);
   pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
   pline.floor := i;
   linelist.Add(pline);
   pline := TlinePoint.Create(nil);
   pline.beginPcode := ((storeyLineList[i])
    [(ij + 1) mod storeyLineList[i].Count]);
   pline.endPcode := ((storeyLineList[i + 1])[ik]);
   pline.beginPoint := fromPcodeGetPoint((storeyLineList[i])
    [(ij + 1) mod storeyLineList[i].Count]);
   pline.endPoint := fromPcodeGetPoint((storeyLineList[i + 1])[ik]);
   pline.floor := i;
   linelist.Add(pline);
   // 三角形中两边替换一边，一种类似最短路径算法中的弗洛伊德算法中的松弛操作 relax operation
   //memo_seaLine.Lines.Add(((storeyLineList[i])[ij]) +
   //  ((storeyLineList[i + 1])[ik]) + '和' + ((storeyLineList[i])
   //  [(ij + 1) mod storeyLineList[i].Count]) + ((storeyLineList[i + 1])
   //  [ik]) + '替换' + ((storeyLineList[i])[ij]) + ((storeyLineList[i])
   //  [(ij + 1) mod storeyLineList[i].Count]));
   for jjj := lineList.Count - 1 downto 0 do
   begin
    //三角形中两边替换一边，将另一边先标记，到时统一删除，系统功能限制，不能立即删除乎
    if (TlinePoint(linelist[jjj]).beginPcode + TlinePoint(
      linelist[jjj]).endPcode) = ((storeyLineList[i])
      [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
      TlinePoint(linelist[jjj]).floor := -111;
    if (TlinePoint(linelist[jjj]).endPcode + TlinePoint(
      linelist[jjj]).beginPcode) = ((storeyLineList[i])
      [ij]) + ((storeyLineList[i])[(ij + 1) mod storeyLineList[i].Count]) then
      TlinePoint(linelist[jjj]).floor := -111;
   end;
   for jjj := pointList.Count - 1 downto 0 do
    //将原先内层高层的点，先标记，到时全删除乎
```

```
        if TPointLine(pointList[jjj]).pcode = ((storeyLineList[i + 1])[ik]) then
          TPointLine(pointList[jjj]).floor := -111;
       //tstringlist(storeyLineList[i])[ij+1].Delete;
       //tstringlist(storeyLineList[i])[ij].Delete;
       TStringList(storeyLineList[i]).insert(ij + 1,
        TStringList(storeyLineList[i + 1])[ik]);
       //三角形中两边替换一边，实际相当于在原来的两点路径中插入一个新点
     end;
     //if jjj = 2 then
     begin
     end;
   end;  //for k
  ////////////////////////////////////////
  // for j := 0 to storeyLineList[i+1].Count - 1 do
  for j := lineList.Count - 1 downto 0 do
   if TlinePoint(linelist[j]).floor = i + 1 then
    //将原先内层高层的边，先标记，到时全删除乎
    TlinePoint(linelist[j]).floor := -111;
  TStringList(storeyLineList[i + 1]).Clear;
  for j := 0 to TStringList(storeyLineList[i]).Count - 1 do
   TStringList(storeyLineList[i + 1]).add(TStringList(storeyLineList[i])[j]);
  TStringList(storeyLineList[i]).Clear;
end;   //i
//新线立即加上，原线标记下次删除之，下面的就是互换的代码，替代不能直接删除而已
tempLineList := TFPList.Create;
// for i := lineList.Count - 1 downto 0 do
//for j := i-1 downto 0 do
//if TlinePoint(linelist[i]).floor <> -111 then
//if TlinePoint(linelist[j]).floor <> -111 then
//if   TlinePoint(linelist[i]).beginPcode+TlinePoint(linelist[i]).endPcode=
//TlinePoint(linelist[j]).beginPcode+TlinePoint(linelist[j]).endPcode  then
//    TlinePoint(linelist[i]).floor:=-111;
//   memo_seaLine.Lines.Add(   TlinePoint(linelist[i]).beginPcode+TlinePoint(linelist[i]).endPcode);
//P91P91:0   P1P1:0  P10P10:0    P100P100:0  如此造成统计点数出错的

for i := lineList.Count - 1 downto 0 do
 if TlinePoint(linelist[i]).floor <> -111 then
  if TlinePoint(linelist[i]).beginPcode = TlinePoint(linelist[i]).endPcode then
    TlinePoint(linelist[i]).floor := -111;
//P91P91:0   P1P1:0  P10P10:0    P100P100:0  如此造成统计点数出错的
for i := lineList.Count - 1 downto 0 do
begin
 try
  if TlinePoint(linelist[i]).floor <> -111 then
  begin
    pline := TlinePoint.Create(nil);
    pline.beginPcode := TlinePoint(linelist[i]).beginPcode;
    pline.endPcode := TlinePoint(linelist[i]).endPcode;
    pline.beginPoint := TlinePoint(linelist[i]).beginPoint;
    pline.endPoint := TlinePoint(linelist[i]).endPoint;
    pline.floor := TlinePoint(linelist[i]).floor;
```

```
      tempLineList.Add(pline);
    end;
  except
  end;
  application.ProcessMessages;
end;
for i := lineList.Count - 1 downto 0 do
begin
  try
    // if TlinePoint(linelist[i]).floor<>-111 then
    if lineList.Items[i] <> nil then
    begin
      TlinePoint(lineList.Items[i]).Free;
      lineList.Items[i] := nil;
      lineList.Count := lineList.Count - 1;
      application.ProcessMessages;
    end;
  except
  end;
  application.ProcessMessages;
end;
lineList.Clear;
for i := tempLineList.Count - 1 downto 0 do
begin
  try
    // if TlinePoint(tempLineList[i]).floor<>-111 then
    begin
      pline := TlinePoint.Create(nil);
      pline.beginPcode := TlinePoint(tempLineList[i]).beginPcode;
      pline.endPcode := TlinePoint(tempLineList[i]).endPcode;
      pline.beginPoint := TlinePoint(tempLineList[i]).beginPoint;
      pline.endPoint := TlinePoint(tempLineList[i]).endPoint;
      pline.floor := TlinePoint(tempLineList[i]).floor;
      LineList.Add(pline);
    end;
  except
  end;
  application.ProcessMessages;
end;
for i := tempLineList.Count - 1 downto 0 do
begin
  try
    // if TlinePoint(linelist[i]).floor<>-111 then
    if tempLineList.Items[i] <> nil then
    begin
      TlinePoint(tempLineList.Items[i]).Free;
      tempLineList.Items[i] := nil;
      tempLineList.Count := lineList.Count - 1;
      application.ProcessMessages;
    end;
  except
```

```
   end;
   application.ProcessMessages;
  end;
  tempLineList.Clear;
  tempLineList.Free;
  for i := pointList.Count - 1 downto 0 do
   TPointLine(pointList[i]).floor := -1;
  onelong := 0.0;
  alllong := 0.0;
  memo_seaLine.Lines.Add('总路径：总点数：' + IntToStr(lineList.Count));
  //P91P91:0   P1P1:0  P10P10:0    P100P100:0  如此造成统计点数出错的
  for i := lineList.Count - 1 downto 0 do
  begin
   try
    onelong := pplong(TlinePoint(linelist[i]).beginPoint, TlinePoint(
     linelist[i]).endPoint);
    memo_seaLine.Lines.Add(TlinePoint(linelist[i]).beginPcode +
     TlinePoint(linelist[i]).endPcode + ':' + floattostr(onelong));
    alllong := alllong + onelong;
    TPointLine(pointList[getpcodeindex(TlinePoint(linelist[i]).beginPcode)]).floor
    := 1;
    TPointLine(pointList[getpcodeindex(TlinePoint(linelist[i]).endPcode)]).floor := 1;
   except
   end;
   application.ProcessMessages;
  end;

  memo_seaLine.Lines.Add('总路长：' + floattostr(alllong));
  ss := '';
  for i := pointList.Count - 1 downto 0 do
   if TPointLine(pointList[i]).floor = -1 then
    ss := ss + TPointLine(pointList[i]).pcode + ',';
  if ss <> '' then
   ShowMessage(ss + '这些点没有被处理');
  //同心圆的中心点暂未处理，因为结果不理想，无兴趣理会
  //sleepli(2);
  //refreshFromLineList(nil);
  refrash.Click;
  //refreshClick(sender);
end;
```